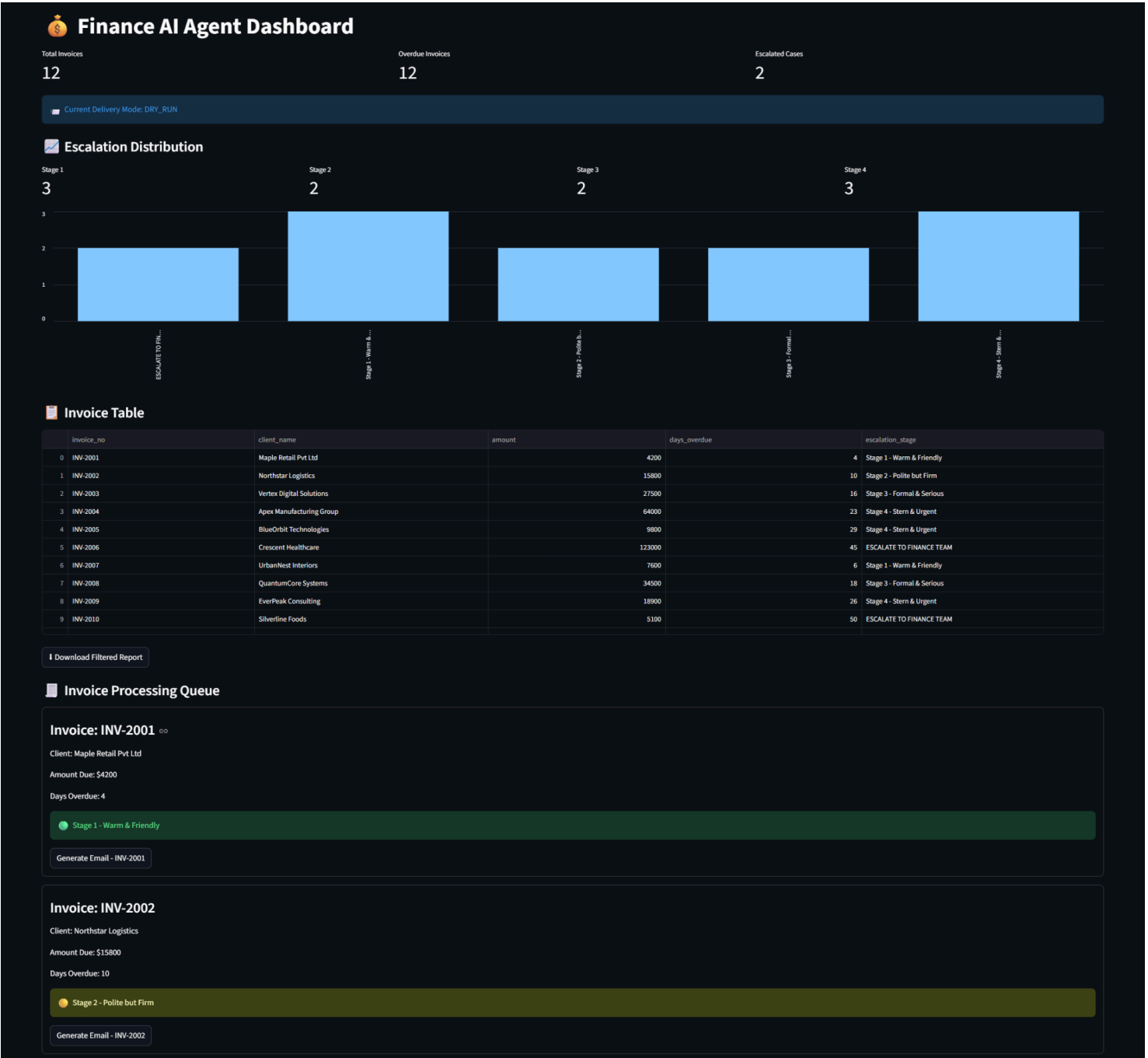


TCI PROJECT

Finance Credit Follow-Up Email Agent

-By Malavika Gupta



INDEX

1. Introduction
2. System Architecture
3. Technical Stack & Decision Log
4. Prompt Engineering & LLM Design
5. Tone Escalation Engine
6. Email Dispatch System
7. Dashboard & Observability
8. Security Risk Mitigation
9. Challenges Faced & Solutions
10. Results & Observations
11. Future Improvements
12. Conclusion
13. References

Introduction

1.1 Project Overview

Accounts receivable and finance teams spend a significant amount of time manually tracking overdue invoices and sending payment reminders to clients. These follow-up processes are often repetitive, inconsistent in tone, and difficult to scale efficiently across a large number of customers. Delayed or inconsistent follow-ups can negatively impact cash flow, increase Days Sales Outstanding (DSO), and create operational inefficiencies within organizations.

This project presents an AI-powered Finance Follow-up Agent designed to automate the process of identifying overdue invoices, generating context-aware payment reminder emails, and dispatching them using progressively escalating communication strategies. The system dynamically adjusts the tone and urgency of each email based on the number of overdue days and prior reminder stages.

The solution combines Large Language Models (LLMs), workflow automation, scheduling, audit logging, and dashboard-based monitoring into a modular workflow system. The agent supports both dry-run simulation mode and real SMTP-based email delivery while maintaining operational safety through escalation limits and structured validation mechanisms.

The project was implemented using Python, Streamlit, APScheduler, Pandas, and a locally hosted Llama 3 model via Ollama.

1.2 Business Problem

Finance departments frequently face challenges in recovering overdue payments from customers. Manual follow-up workflows require employees to repeatedly:

- monitor unpaid invoices,
- calculate overdue durations,
- determine escalation severity,
- draft reminder emails,
- maintain communication records, and
- escalate unresolved cases for further review.

These processes are time-consuming and prone to inconsistency. Different team members may communicate with varying levels of urgency, resulting in unstructured client interactions and inefficient collections workflows.

In many organizations, delayed payment follow-ups contribute to increased Days Sales Outstanding (DSO), reduced operational liquidity, and unnecessary administrative overhead.

An intelligent workflow automation system can help standardize communication, automate repetitive tasks, reduce manual effort, and ensure that overdue cases are escalated consistently according to predefined business rules.

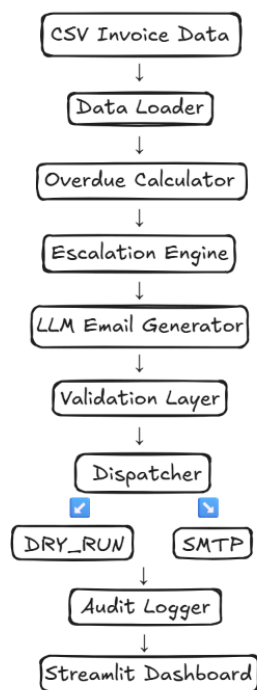
1.3 Objectives of the System

The primary objective of this project was to design and implement an AI-powered workflow agent capable of automating invoice follow-up operations for finance teams.

The system was designed with the following goals:

- Automatically ingest invoice records from structured data sources.
- Detect overdue invoices and calculate escalation stages dynamically.
- Generate personalized follow-up emails using a Large Language Model (LLM).
- Progressively escalate communication tone based on overdue duration.
- Support both simulated (dry-run) and real SMTP email delivery.
- Maintain audit logs for all generated communications and dispatch events.
- Prevent uncontrolled escalation by routing severely overdue cases to human review.
- Provide operational visibility through an interactive Streamlit dashboard.
- Ensure safer AI behavior through structured outputs, validation layers, and security-conscious design decisions.

The final system functions as a modular workflow automation prototype that combines AI-driven language generation with practical operational controls.



System Architecture

2.1 High-Level Workflow

The Finance AI Agent was designed as a modular workflow automation system consisting of multiple independent processing components. The architecture follows a sequential pipeline where invoice records are processed through validation, escalation analysis, AI-driven email generation, dispatch handling, and audit logging.

The system workflow begins by loading invoice data from a structured CSV dataset. The agent then calculates the overdue duration for each invoice and maps it to an escalation stage using predefined business rules.

For invoices eligible for automated follow-up, the system invokes a Large Language Model (LLM) to generate personalized payment reminder emails with tone and urgency dynamically adjusted according to the escalation stage.

Generated emails are then routed through a dispatcher layer, which determines whether the email should be:

- simulated in dry-run mode, or
- sent through a real SMTP email service.

Every action performed by the agent is recorded in an audit logging system for traceability and operational monitoring. Cases exceeding the maximum escalation threshold are automatically flagged for manual finance or legal review instead of receiving further automated communication.

The system also includes:

- a Streamlit-based dashboard for operational visibility, and
- an APScheduler-based automation layer for periodic invoice processing.

2.2 Agent Pipeline Overview

The Finance AI Agent operates through a staged processing pipeline designed to separate business logic, AI generation, dispatch handling, and monitoring responsibilities.

The processing flow is structured as follows:

1. **Data Ingestion Layer**
Invoice records are loaded from a CSV dataset using Pandas.
2. **Overdue Analysis Layer**
The system calculates overdue durations by comparing invoice due dates against the current system date.

-
3. **Escalation Engine**
Based on overdue duration, invoices are mapped to predefined escalation stages ranging from friendly reminders to manual escalation review.
 4. **LLM Email Generation Layer**
The LLM generates context-aware payment reminder emails using stage-specific prompting strategies and structured formatting constraints.
 5. **Validation & Parsing Layer**
Generated outputs are validated and parsed into structured objects before downstream processing.
 6. **Dispatch Layer**
Emails are routed either to:
 - dry-run simulation mode, or
 - real SMTP delivery mode.
 7. **Audit Logging Layer**
All system actions, including generated emails and dispatch outcomes, are logged for traceability.
 8. **Dashboard & Monitoring Layer**
A Streamlit dashboard provides operational visibility into invoice queues, escalation stages, generated emails, and audit history.
 9. **Automation Scheduler**
APScheduler periodically triggers autonomous invoice processing workflows.

2.3 Component Breakdown

Data Loader

The data loader module is responsible for reading invoice records from structured CSV files using the Pandas library. The module converts due date fields into datetime objects to support overdue duration calculations and downstream processing.

Escalation Engine

The escalation engine applies predefined business rules to determine the appropriate reminder stage for each invoice. The escalation logic progressively increases urgency based on overdue duration while enforcing a hard escalation cap for severely overdue invoices.

Email Generator

The email generator module interacts with the Large Language Model to create personalized payment reminder emails. The system uses stage-specific prompting strategies, structured formatting instructions, and behavioral constraints to ensure consistent escalation tone and output formatting.

Dispatcher

The dispatcher module abstracts the email delivery process by routing generated emails to either:

- dry-run simulation mode, or
- real SMTP-based delivery.

This abstraction enables safe testing and flexible deployment configurations.

Audit Logger

The audit logging system records all workflow events, including:

- invoice identifiers,
- escalation stages,
- delivery mode,
- timestamps, and
- dispatch status.

Logs are stored in JSONL format to support traceability and debugging.

Scheduler

The APScheduler-based scheduler periodically executes invoice processing workflows automatically, enabling autonomous monitoring and follow-up execution without manual intervention.

Dashboard

The Streamlit dashboard provides an interactive operational interface for viewing invoice queues, escalation distributions, generated emails, delivery modes, and audit logs.

Technical Stack & Decision Log

3.1 Programming Language

The project was implemented in Python due to its strong ecosystem for:

- AI integration,
- workflow automation,
- data processing, and
- rapid prototyping.

Python libraries such as Pandas, Streamlit, APScheduler, and Pydantic enabled fast development of modular workflow components.

3.2 LLM Model Chosen

Model - Llama 3

Provider - Meta

Runtime - Ollama (local inference)

Llama 3 was selected because it:

- supports strong instruction-following,
- performs well for structured text generation,
- can run locally through Ollama,
- avoids external API costs during development,
- reduces dependency on cloud APIs, and
- improves privacy by keeping invoice data local.

Local inference was preferred over cloud-hosted APIs due to quota limitations encountered during development and to reduce exposure of financial data to third-party services.

3.3 Agent Framework Used

Instead of using a heavy multi-agent orchestration framework, the project uses a modular custom-agent workflow architecture implemented directly in Python.

The system follows a sequential workflow pipeline consisting of:

1. data ingestion,
2. overdue analysis,
3. escalation mapping,
4. LLM email generation,

-
5. validation,
 6. dispatch handling, and
 7. audit logging.

This approach was chosen because:

- the workflow requirements were deterministic,
- a lightweight architecture simplified debugging,
- modular components were easier to test independently, and
- the project did not require complex multi-agent coordination.

3.4 Dashboard Framework

Streamlit was used to build the operational dashboard because it enables rapid development of interactive data-driven interfaces directly in Python.

The dashboard provides:

- invoice visibility,
- escalation filtering,
- generated email previews,
- audit log monitoring, and
- delivery mode tracking.

3.5 Scheduling Framework

APScheduler was used to automate periodic invoice processing workflows.

The scheduler enables the system to:

- monitor overdue invoices automatically,
- trigger follow-up workflows periodically, and
- simulate autonomous agent behavior.

3.6 Logging System

The system uses JSONL-based audit logging to record:

- invoice identifiers,
- escalation stages,
- timestamps,
- dispatch status, and
- delivery mode.

Structured logging improves:

- traceability,
- debugging,
- observability, and
- workflow auditing.

Prompt Engineering & LLM Design

4.1 Initial Prompting Challenges

Initial prompt designs produced several issues:

- inconsistent output formatting,
- malformed JSON responses,
- overly verbose emails,
- weak differentiation between escalation stages, and
- occasional repetition in generated text.

Early prompts behaved more like general AI assistant instructions rather than finance-oriented workflow prompts.

4.2 Prompt Evolution

The prompting strategy evolved iteratively during development.

The following improvements were introduced:

- stage-specific personas,
- stricter formatting instructions,
- hard output constraints,
- negative prompting,
- CTA enforcement, and
- structured parsing validation.

Instead of using a single generic prompt, the system dynamically applies different behavioral instructions depending on the escalation stage.

4.3 Final Prompt Architecture

The final prompting architecture uses:

- role conditioning,
- escalation-specific personas,
- formatting rules, and
- constrained output structure.

Each escalation stage was assigned a dedicated communication profile:

Stage	Tone
Stage 1	Warm & Friendly
Stage 2	Polite but Firm
Stage 3	Formal & Serious
Stage 4	Stern & Urgent

The prompts explicitly restrict:

- filler phrases,
- excessive politeness,
- generic AI assistant language, and
- inconsistent formatting.

The model is instructed to generate concise operational finance emails rather than conversational chatbot responses.

4.4 Structured Output & Validation

To improve reliability, the generated outputs were parsed into structured fields:

- subject,
- tone, and
- body.

Validation and parsing layers were added to:

- detect malformed outputs,
- prevent workflow crashes,
- improve consistency, and
- reduce hallucinated formatting.

This significantly improved output stability during batch processing.

Tone Escalation Engine

5.1 Escalation Logic

The system uses a rule-based escalation engine to determine the communication tone and urgency level for each overdue invoice.

Escalation stages are assigned based on the number of overdue days calculated from the invoice due date.

Overdue Days	Stage	Tone
1-7	Stage 1	Warm & Friendly
8-14	Stage 2	Polite but Firm
15-21	Stage 3	Formal & Serious
22-30	Stage 4	Stern & Urgent
30+	Escalation	Manual Finance Review

The escalation logic ensures that communication pressure increases progressively while maintaining professional language.

5.2 Stage 1 – Warm & Friendly

Stage 1 emails assume that the missed payment may have been accidental or overlooked.

The communication style:

- remains polite,
- avoids aggressive wording,
- includes payment details, and
- encourages quick resolution.

The objective is to recover payment while preserving positive client relationships.

5.3 Stage 2 – Polite but Firm

Stage 2 emails increase urgency while maintaining professionalism.

The communication:

- acknowledges that payment is still pending,
- requests confirmation of a payment timeline, and
- introduces stronger accountability language.

The goal is to obtain a payment commitment from the client.

5.4 Stage 3 – Formal & Serious

Stage 3 emails formally communicate escalating concern regarding delayed payment.

The system:

- references previous reminders,
- warns about potential impact on credit terms, and
- requests a response within 48 hours.

The communication becomes more procedural and compliance-oriented.

5.5 Stage 4 – Stern & Urgent

Stage 4 emails function as final reminders before escalation.

The communication:

- emphasizes urgency,
- warns about further recovery action, and
- demands immediate response or payment.

The language is intentionally direct while remaining professional.

5.6 Escalation Cap

Invoices overdue beyond 30 days are not processed through automated email generation.

Instead, the system:

- flags the invoice for manual finance/legal review, and
- prevents uncontrolled automated escalation.

This safeguard ensures that high-risk financial cases remain under human supervision.

Email Dispatch & Automation

6.1 Delivery Modes

The system supports two configurable delivery modes:

Mode	Description
DRY_RUN	Simulates email delivery without sending real emails
SMTP	Sends real emails through Gmail SMTP integration

The active mode is controlled through environment variables, allowing safe switching between testing and production behavior without modifying source code.

6.2 Dispatcher Architecture

A dispatcher layer was implemented to abstract email delivery logic from the rest of the workflow pipeline.

Instead of directly calling SMTP or simulation functions, the system routes generated emails through a centralized dispatch module. This improves:

- modularity,
- maintainability, and
- safe testing.

The dispatcher dynamically selects the appropriate delivery method based on the configured runtime environment.

6.3 SMTP Integration

SMTP-based delivery was implemented using Gmail SMTP services and Python's `smtplib` library.

Authentication credentials were stored securely using environment variables and loaded through `python-dotenv`.

Transport Layer Security (TLS) was enabled before authentication to encrypt communication with the SMTP server.

Dashboard & Monitoring

7.1 Streamlit Dashboard

A Streamlit-based dashboard was developed to provide operational visibility into the workflow system.

The dashboard enables users to:

- monitor invoice queues,
- review escalation stages,
- generate emails interactively,
- inspect audit logs, and
- monitor dispatch status.

7.2 Dashboard Features

The dashboard includes:

- escalation-stage filtering,
- invoice summary metrics,
- escalation distribution metrics,
- expandable email previews,
- audit log viewing, and
- delivery mode indicators.

Color-coded escalation badges were added to improve urgency visibility and workflow readability.

7.3 Human-in-the-Loop Design

Although the system supports automation, the dashboard also enables manual review and interaction.

This hybrid workflow design allows:

- human oversight,
- safer testing,
- controlled dispatch behavior, and
- operational transparency.

The approach reduces the risk of uncontrolled fully autonomous communication workflows.

Security Risk Mitigation

8.1 Prompt Injection Mitigation

To reduce the risk of prompt injection and output manipulation:

- invoice inputs were treated as structured data rather than executable instructions,
- prompts used constrained formatting rules,
- generated outputs were parsed into structured fields, and
- validation layers were added before downstream processing.

The system also avoided allowing arbitrary user-generated prompt modifications during runtime.

8.2 Data Privacy & PII Protection

The system was designed to minimize unnecessary exposure of financial data.

Key measures included:

- local LLM inference through Ollama,
- avoiding cloud-based processing during development,
- limiting stored audit information to operational fields only, and
- preventing sensitive credentials from appearing in logs.

This reduced external transmission of invoice-related information.

8.3 API Key & Credential Protection

Sensitive credentials such as:

- SMTP credentials, and
- API configuration variables

were stored using `.env` files and loaded through `python-dotenv`.

Credentials were:

- excluded from version control using `.gitignore`,
- never hardcoded into source files, and
- separated from application logic.

8.4 Hallucination Mitigation

Several safeguards were added to reduce hallucinated or inconsistent outputs:

-
- structured prompting,
 - stage-specific constraints,
 - validation layers,
 - parser checks, and
 - deterministic business-rule escalation logic.

The model was restricted to generating communication content only, while workflow decisions remained rule-based.

8.5 Unauthorized Access Prevention

The current prototype operates locally and does not expose public APIs or externally accessible endpoints.

For production deployment, additional protections such as:

- authentication,
- role-based access control,
- API authorization, and
- rate limiting

would be recommended.

8.6 Email Spoofing Prevention

Real email delivery was implemented using authenticated SMTP-based sending from verified accounts.

The project primarily operated in DRY_RUN mode during development to prevent accidental unintended email delivery.

For production systems, additional protections such as:

- SPF,
- DKIM, and
- DMARC

would be recommended to improve sender verification and reduce spoofing risks.

8.7 Safe Testing Architecture

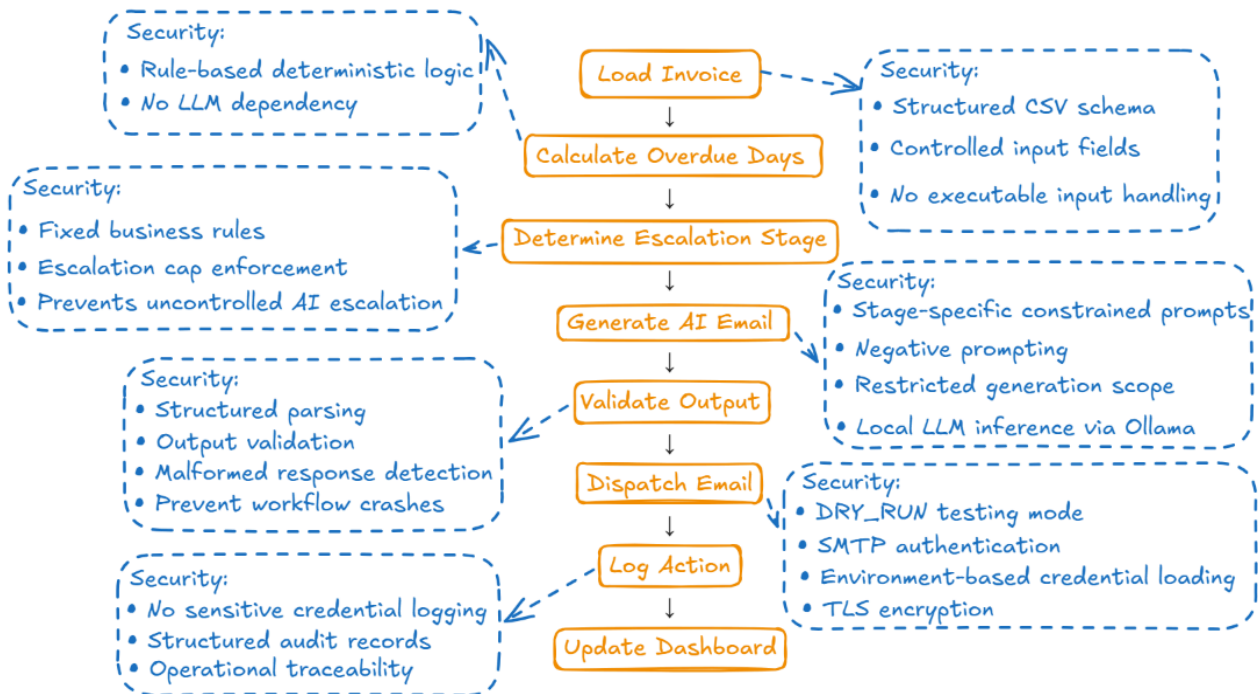
A dedicated DRY_RUN mode was implemented to safely test workflow execution without sending real emails.

This design:

- reduced accidental delivery risk,

- simplified debugging,
- improved testing safety, and
- enabled controlled development iteration.

The dispatcher architecture allowed switching between simulated and real delivery modes through environment configuration rather than source code changes.



Challenges Faced & Solutions

9.1 API Quota & Model Access Issues

Initial development attempted to use cloud-hosted LLM APIs. However, API quota limitations and billing restrictions interrupted testing workflows.

To resolve this issue, the system was migrated to a locally hosted Llama 3 model using Ollama, which eliminated API cost concerns, improved privacy, and enabled unrestricted local experimentation.

9.2 Output Formatting Instability

Early prompt versions produced malformed outputs and inconsistent formatting, causing JSON parsing failures during batch processing.

The issue was addressed through:

- stricter prompt constraints,
- structured output formatting,
- validation layers, and parser improvements.

9.3 Weak Tone Differentiation

Initial prompts generated emails with insufficient escalation differences between reminder stages.

This was improved by introducing:

- stage-specific personas,
- escalation-aware instructions,
- negative prompting, and
- stricter behavioral constraints.

9.4 SMTP Safety Concerns

Real email integration introduced risks related to accidental email delivery during development.

To reduce this risk:

- DRY_RUN mode was implemented,
- delivery mode abstraction was added through the dispatcher layer, and
- SMTP testing was initially restricted to self-addressed emails only.

Results & Observations

10.1 Workflow Execution Results

The final system successfully:

- processed overdue invoice records,
- assigned escalation stages dynamically,
- generated personalized payment reminder emails,
- supported both dry-run and SMTP delivery modes,
- logged workflow actions, and
- flagged high-risk invoices for manual review.

The workflow remained stable during batch processing across multiple escalation scenarios.

10.2 Tone Escalation Results

The final prompting architecture produced clearly differentiated communication styles across escalation stages.

Observed behavior included:

- softer language during early reminders,
- progressively stronger accountability language,
- formal escalation wording for serious cases, and
- urgent final-warning messaging before manual escalation.

The generated emails became significantly more concise and operational after prompt refinement iterations.

10.3 System Reliability

System reliability improved substantially after introducing:

- structured prompting,
- validation layers,
- parsing safeguards, and
- modular workflow separation.

The final implementation showed:

- stable batch execution,
- reduced malformed outputs, and
- improved formatting consistency.

10.4 Operational Observations

Several practical observations emerged during development:

- lightweight modular architectures simplified debugging,
- local LLM inference improved experimentation flexibility,
- dry-run execution significantly reduced testing risk,
- prompt constraints strongly affected output consistency, and
- separating business logic from AI generation improved system reliability.

The project demonstrated that workflow orchestration and operational safeguards are equally important as the LLM itself in automation systems.

Future Improvements

11.1 Retry Protection

Future versions could include retry protection mechanisms to prevent repeated reminders from being sent within short intervals during scheduled execution.

11.2 Database Integration

The current CSV-based workflow could be extended using:

- SQL databases,
- ERP integrations, or
- CRM platforms

to support larger-scale production environments.

11.3 Advanced Analytics

Additional dashboard analytics could include:

- payment recovery trends,
- escalation statistics,
- invoice aging charts, and
- delivery success monitoring.

11.4 Multi-Channel Communication

Future versions could support:

- SMS reminders,
- WhatsApp notifications,
- CRM ticket generation, and
- multilingual communication workflows.

11.5 Retrieval-Augmented Context

The system could be extended using Retrieval-Augmented Generation (RAG) to incorporate:

- customer payment history,
- prior communication records, and
- account-level risk information

during email generation.

Conclusion

This project demonstrated the design and implementation of an AI-powered finance workflow automation system for overdue invoice follow-ups.

The final system combined:

- rule-based workflow orchestration,
- LLM-driven communication generation,
- scheduling automation,
- audit logging,
- dashboard monitoring, and
- configurable email delivery modes

within a modular architecture.

The project also highlighted the importance of:

- prompt engineering,
- operational safeguards,
- structured validation,
- security-conscious design, and
- human oversight

when building practical AI workflow systems.

The final prototype successfully automated repetitive finance follow-up tasks while maintaining controlled escalation behavior and operational transparency.

References

13.1 Libraries & Frameworks

1. [Pandas Documentation](#)
2. [Streamlit Documentation](#)
3. [APScheduler Documentation](#)
4. [Pydantic Documentation](#)
5. [Python smtplib Documentation](#)

13.2 LLM & AI References

6. Meta Llama 3
7. [Ollama](#)

13.3 Security & Email Standards

8. [OWASP Prompt Injection Overview](#)
9. [Google SMTP Documentation](#)
10. [DMARC Organization](#)
11. [DKIM Overview](#)
12. [SPF Record Syntax](#)